

03 - Odoo Deployment: GitHub Source Code

LIONGATE SARL - Technical Runbook

Version 1.0 - Initial release

Table of Contents

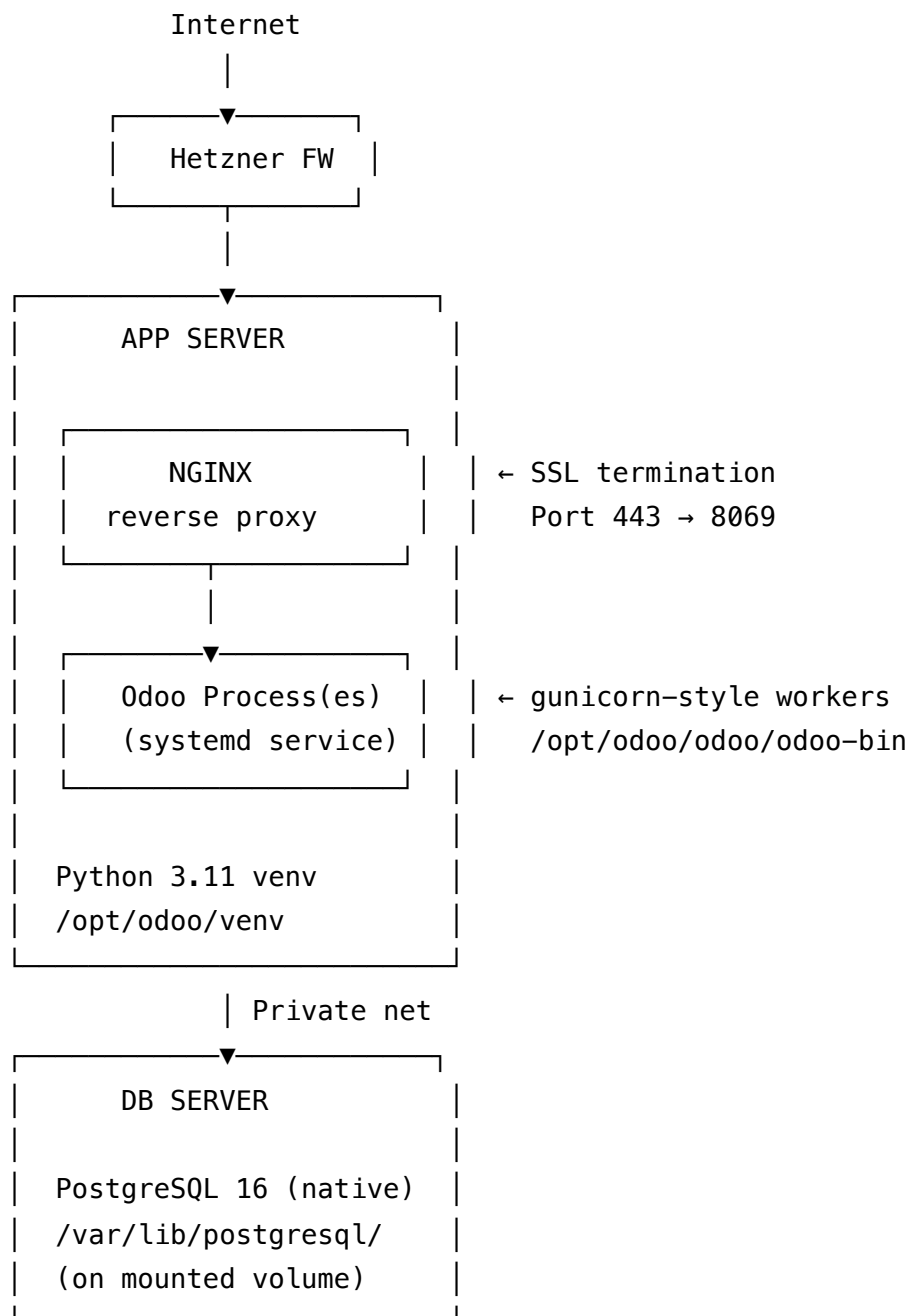
1. [Overview & Architecture](#)
2. [Docker vs Source: Comparison](#)
3. [Version Assumptions](#)
4. [Local Development Setup \(Source\)](#)
5. [Hetzner Server Provisioning](#)
6. [OS Preparation & Dependencies](#)
7. [Python Environment Setup](#)
8. [PostgreSQL Setup \(Native\)](#)
9. [Odoo Source Checkout](#)
10. [Addons Path Configuration](#)
11. [Odoo Config File Setup](#)
12. [System Service Setup \(systemd\)](#)
13. [Reverse Proxy with Nginx](#)
14. [SSL/TLS Configuration](#)
15. [Domain Mapping](#)
16. [Performance Tuning](#)
17. [Production Hardening](#)
18. [Custom Addons Strategy](#)
19. [Backup Strategy](#)
20. [Monitoring & Logging](#)
21. [Upgrade Path](#)
22. [Recovery & Restore](#)
23. [Validation & Testing](#)
24. [Production Deployment Checklist](#)

Overview & Architecture

What This Document Covers

This runbook describes deploying Odoo from the **official GitHub source code** (github.com/odoo/odoo) directly on the OS - without Docker. It covers local development, Hetzner production deployment, and full operational procedures.

Architecture Overview



Technology Stack

Component	Technology	Version
Application	Odoo (source)	19.0
Python	CPython	3.11.x
Runtime env	Python virtualenv	system
Database	PostgreSQL	16.x
Reverse proxy	Nginx	1.24+
Service manager	systemd	OS default
SSL	Let's Encrypt / Certbot	Latest
OS	Ubuntu	22.04 LTS

Docker vs Source: Comparison

Dimension	Docker Image	Source Code
Setup time	Fast (minutes)	Slow (1–2 hours)
Dependency management	Handled by image	Manual - you install deps
Python version	Fixed by image	You control
Custom deps	Requires Dockerfile rebuild	Direct <code>pip install</code>
Odoo source access	No (pre-compiled)	Full source
Patching / hotfixes	<code>git pull</code> in custom Dockerfile	<code>git pull</code> directly
Upgrade complexity	Pull new tag	<code>git checkout 19.0</code> , reinstall deps
Debugging	Through container shell	Direct - edit files, restart

Dimension	Docker Image	Source Code
OCA module compatibility	Easier - just mount	Must manage venv deps manually
Production isolation	Strong (containers)	Moderate (venv + systemd)
Resource overhead	Slightly higher (Docker)	Minimal
Team familiarity	Most teams know Docker	Requires Linux/Python knowledge
CI/CD integration	Easy (docker build/push)	More complex
Recommended for	Most projects	Deep customization / patch work

When to Choose Source Deployment

Use source deployment when:

- You need to apply patches directly to Odoo core
- You are maintaining a heavily customized Odoo that diverges from upstream
- You are debugging complex OCA module compatibility issues
- The team has strong Linux/Python skills and prefers direct control
- You want minimal container overhead on low-RAM servers

Default recommendation: Use Docker (see `02-odoo-deployment-docker-image.md`) unless one of the above applies.

Version Assumptions

Item	Value
Odoo version	19.0 branch
PostgreSQL	16
Python	3.11
Ubuntu	22.04 LTS

Item	Value
Odoo user	odoo (system user, no login shell)
Odoo root	/opt/odoo
Venv	/opt/odoo/venv
Source	/opt/odoo/odoo (git repo)
OCA modules	/opt/odoo/oca
Custom modules	/opt/odoo/custom
Config	/etc/odoo/odoo.conf
Data dir	/var/lib/odoo
Logs	/var/log/odoo/odoo.log
PID	/var/run/odoo/odoo.pid

Local Development Setup (Source)

Prerequisites

```
# Ubuntu 22.04 / Debian 12 / macOS
python3 --version    # Must be 3.11+
psql --version       # Must be 14+
git --version
```

Local Setup Steps

1. Create local directory

```
mkdir -p ~/dev/odoo-source && cd ~/dev/odoo-source
```

2. Clone Odoo

```
git clone https://github.com/odoo/odoo.git --branch 19.0 --depth 1
cd odoo
```

3. Create Python virtualenv

```
python3 -m venv venv
source venv/bin/activate
```

4. Install Python dependencies

```
pip install --upgrade pip wheel setuptools
pip install -r requirements.txt
```

5. Install wkhtmltopdf (for PDF reports)

Ubuntu:

```
wget https://github.com/wkhtmltopdf/packaging/releases/download/0.12.6.1-3/wkhtmltox_0.12.6.1-3.jammy_amd64.deb
sudo dpkg -i wkhtmltox_0.12.6.1-3.jammy_amd64.deb
sudo apt-get -f install -y
```

6. Set up local PostgreSQL (if not running)

```
sudo -u postgres createuser -s odoo_dev
sudo -u postgres createdb odoo_dev -O odoo_dev
```

7. Create odoo.conf

```
cat > ~/dev/odoo-source/odoo.conf << 'EOF'
[options]
addons_path = /home/youruser/dev/odoo-source/odoo/addons,/home/youruser/dev/odoo-source
data_dir = /home/youruser/dev/odoo-source/data
db_host = localhost
db_port = 5432
db_user = odoo_dev
db_password = False
db_name = odoo_dev
log_level = debug
http_port = 8069
admin_passwd = admin_local
EOF
```

```
# 8. Initialize and start
./odoo-bin --config=../odoo.conf --dev=all
```

Access: <http://localhost:8069>

Hetzner Server Provisioning

Infrastructure for Source Deployment

Same server requirements as Docker deployment:

Resource	Specification
App server	CX42 (8 vCPU, 16 GB RAM)
DB server	CX32 (4 vCPU, 8 GB RAM)
DB Volume	100 GB
Backup	Storage Box BX31
Floating IP	Yes
Private network	10.0.0.0/16

Refer to `01-hosting-platform-billing-and-infrastructure-strategy.md` for provisioning steps.

OS Preparation & Dependencies

Run on the **App Server**.

```
ssh root@<app-server-ip>
```

```
# Update system
```

```
apt update && apt upgrade -y
```

```
# Set timezone
```

```
timedatectl set-timezone Africa/Douala
```

```
# Install system dependencies for Odoo 17
```

```
apt install -y \
```

```
python3.11 python3.11-dev python3.11-venv python3-pip \
```

```
build-essential libssl-dev libffi-dev \
```

```
libpq-dev libldap2-dev libsasl2-dev \
```

```
libjpeg-dev libpng-dev zlib1g-dev \
```

```
libreoffice \
```

```
fontconfig \
```

```
git curl wget unzip \
```

```
nginx \
```

```
ufw fail2ban \
```

```
supervisor \
```

```
postgresql-client-16 \
```

```
nodejs npm
```

```
# Install wkhtmltopdf (required for PDF generation in Odoo)
```

```
wget https://github.com/wkhtmltopdf/packaging/releases/download/0.12.6.1-3/wkhtmltox_0.
```

```
apt install -y ./wkhtmltox_0.12.6.1-3.jammy_amd64.deb
```

```
rm wkhtmltox_0.12.6.1-3.jammy_amd64.deb
```

```
# Verify
```

```
wkhtmltopdf --version
```

```
# Create odoo system user (no login shell)
```

```
adduser --system --home /opt/odoo --group --shell /bin/bash odoo
```

```
# Create required directories
```

```
mkdir -p /etc/odoo
```

```
mkdir -p /var/lib/odoo
```

```
mkdir -p /var/log/odoo
```

```
mkdir -p /var/run/odoo
```

```
# Set ownership
```



```
chown odoo:odoo /var/lib/odoo /var/log/odoo /var/run/odoo
chmod 750 /var/lib/odoo /var/log/odoo
```

Firewall Setup (App Server)

```
ufw default deny incoming
ufw default allow outgoing
ufw allow from YOUR_OFFICE_IP to any port 22
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable
```

Python Environment Setup

```
# Switch to odoo user
su - odoo

# Create virtualenv in Odoo home
python3.11 -m venv /opt/odoo/venv

# Activate venv
source /opt/odoo/venv/bin/activate

# Upgrade pip and tooling
pip install --upgrade pip wheel setuptools

# Verify
python --version    # Should show Python 3.11.x
which pip           # Should be /opt/odoo/venv/bin/pip
```

PostgreSQL Setup (Native)

Run on the **DB Server**.

```
ssh root@<db-server-ip>
```

```
# Install PostgreSQL 16
```

```
apt install -y curl ca-certificates
```

```
curl https://www.postgresql.org/media/keys/ACCC4CF8.asc | gpg --dearmor -o /usr/share/k
```

```
echo "deb [signed-by=/usr/share/keyrings/postgresql-keyring.gpg] http://apt.postgresql.
```

```
apt update
```

```
apt install -y postgresql-16 postgresql-client-16
```

```
# Move PostgreSQL data to mounted volume
```

```
systemctl stop postgresql
```

```
# Format and mount volume (first time only)
```

```
mkfs.ext4 /dev/sdb
```

```
mkdir -p /mnt/pgdata
```

```
mount /dev/sdb /mnt/pgdata
```

```
echo "/dev/sdb /mnt/pgdata ext4 defaults 0 0" >> /etc/fstab
```

```
# Move data
```

```
rsync -av /var/lib/postgresql/ /mnt/pgdata/
```

```
rm -rf /var/lib/postgresql
```

```
ln -s /mnt/pgdata /var/lib/postgresql
```

```
# Update PostgreSQL to use new data dir
```

```
# Edit /etc/postgresql/16/main/postgresql.conf:
```

```
# data_directory = '/mnt/pgdata/16/main'
```

```
systemctl start postgresql
```

```
systemctl enable postgresql
```

```
# Create odoo DB user
```

```
sudo -u postgres psql << 'EOF'
```

```
CREATE USER odoo_prod WITH PASSWORD 'REPLACE_WITH_STRONG_PASSWORD';
```

```
ALTER USER odoo_prod CREATEDB;
```

```
CREATE DATABASE odoo_production OWNER odoo_prod;
```

```
EOF
```

```
# Allow app server connection (in pg_hba.conf)
```

```
echo "host odoo_production odoo_prod 10.0.1.10/32 md5" >> /etc/postgresql/16/main/pg_hb
```

```
# Edit postgresql.conf to listen on private network
```

```
sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '10.0.1.20,localhost'/" \
/etc/postgresql/16/main/postgresql.conf
```

```
systemctl reload postgresql
```

PostgreSQL Tuning (/etc/postgresql/16/main/postgresql.conf)

```
# For CX32 (8 GB RAM)
max_connections = 100
shared_buffers = 2GB
effective_cache_size = 6GB
maintenance_work_mem = 512MB
checkpoint_completion_target = 0.9
wal_buffers = 64MB
default_statistics_target = 100
random_page_cost = 1.1
work_mem = 52428kB
min_wal_size = 1GB
max_wal_size = 4GB
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers = 4
```

```
systemctl restart postgresql
```

Odoo Source Checkout

On the **App Server**, as the `odoo` user:

```
su - odoo
cd /opt/odoo

# Clone Odoo 19.0 (production - shallow to save disk)
git clone https://github.com/odoo/odoo.git \
  --branch 19.0 \
  --single-branch \
  --depth 1 \
  odoo

# Clone OCA dependencies (example)
# git clone https://github.com/OCA/partner-contact.git \
#   --branch 19.0 --depth 1 oca/partner-contact

# Create custom addons directory
mkdir -p /opt/odoo/custom

# Activate venv and install Odoo requirements
source /opt/odoo/venv/bin/activate
pip install -r /opt/odoo/odoo/requirements.txt

# Install additional OCA/third-party requirements as needed
# pip install python-stdnum phonenumbers vobject

# Verify Odoo binary
/opt/odoo/venv/bin/python /opt/odoo/odoo/odoo-bin --version
```

Addons Path Configuration

Odoo's `addons_path` is a comma-separated list of directories Odoo scans for modules.

Standard Addons Path Layout

```
/opt/odoo/  
├─ odoo/                               ← Core Odoo modules  
│   └─ addons/  
├─ oca/  
│   ├── partner-contact/              ← OCA community modules  
│   └─ account-financial-tools/  
└─ custom/                             ← LIONGATE custom modules  
    └─ custom_crm_fields/
```

Path Setting (in odoo.conf)

```
addons_path = /opt/odoo/odoo/addons,/opt/odoo/oca,/opt/odoo/custom
```

Note: Each directory in the path is scanned recursively for modules (`__manifest__.py` marker).
Keep OCA modules in separate subdirectories within `/opt/odoo/oca/` .

Odoo Config File Setup

/etc/odoo/odoo.conf

```
[options]
; --- Paths ---
addons_path = /opt/odoo/odoo/addons,/opt/odoo/oca,/opt/odoo/custom
data_dir = /var/lib/odoo

; --- Database ---
db_host = 10.0.1.20
db_port = 5432
db_user = odoo_prod
db_password = REPLACE_WITH_STRONG_PASSWORD
db_name = odoo_production
db_maxconn = 64

; --- Security ---
admin_passwd = REPLACE_WITH_STRONG_MASTER_PASSWORD
list_db = False

; --- HTTP ---
http_port = 8069
http_interface = 127.0.0.1
longpolling_port = 8072

; --- Proxy ---
proxy_mode = True

; --- Workers ---
workers = 6
max_cron_threads = 2

; --- Memory limits ---
limit_memory_hard = 2684354560
limit_memory_soft = 2147483648
limit_request = 8192
limit_time_cpu = 600
limit_time_real = 1200
limit_time_real_cron = -1

; --- Logging ---
```

```
log_level = warn
log_handler = :WARNING
logfile = /var/log/odoo/odoo.log
log_rotate = True

; --- Email ---
smtp_server = smtp-relay.brevo.com
smtp_port = 587
smtp_ssl = False
smtp_starttls = True
smtp_user = your-smtp-login
smtp_password = REPLACE_WITH_SMTP_SECRET
email_from = noreply@clientname.com

; --- Session ---
session_db_passwd = REPLACE_WITH_SESSION_SECRET

; --- Misc ---
without_demo = True
server_wide_modules = base,web

# Set permissions
chown root:odoo /etc/odoo/odoo.conf
chmod 640 /etc/odoo/odoo.conf
```

System Service Setup (systemd)

/etc/systemd/system/odoo.service

[Unit]

Description=Odoo 19.0

Requires=network.target

After=network.target postgresql.service

[Service]

Type=simple

SyslogIdentifier=odoo

PermissionsStartOnly=true

User=odoo

Group=odoo

ExecStart=/opt/odoo/venv/bin/python /opt/odoo/odoo/odoo-bin \
--config /etc/odoo/odoo.conf \
--logfile /var/log/odoo/odoo.log

ExecReload=/bin/kill -HUP \$MAINPID

KillMode=mixed

StandardOutput=journal+console

StandardError=journal+console

Restart=on-failure

RestartSec=5s

; Runtime directory and permissions

RuntimeDirectory=odoo

RuntimeDirectoryMode=0755

; Environment

Environment="HOME=/opt/odoo"

Environment="PATH=/opt/odoo/venv/bin:/usr/local/bin:/usr/bin:/bin"

[Install]

WantedBy=multi-user.target


```
# Reload systemd and enable service
systemctl daemon-reload
systemctl enable odoo
systemctl start odoo
systemctl status odoo

# View logs
journalctl -u odoo -f
tail -f /var/log/odoo/odoo.log
```

Log Rotation

```
# /etc/logrotate.d/odoo
cat > /etc/logrotate.d/odoo << 'EOF'
/var/log/odoo/odoo.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
        systemctl restart odoo > /dev/null 2>&1 || true
    endscript
}
EOF
```

Reverse Proxy with Nginx

```
# Install Nginx
apt install -y nginx
```

/etc/nginx/sites-available/odoo

```
upstream odoo {
    server 127.0.0.1:8069;
}

upstream odoo_chat {
    server 127.0.0.1:8072;
}

server {
    listen 80;
    server_name erp.clientname.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name erp.clientname.com;

    ssl_certificate      /etc/letsencrypt/live/erp.clientname.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/erp.clientname.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers off;
    ssl_session_cache shared:SSL:10m;
    ssl_stapling on;
    ssl_stapling_verify on;
    add_header Strict-Transport-Security "max-age=63072000" always;
    add_header X-Frame-Options SAMEORIGIN;
    add_header X-Content-Type-Options nosniff;

    access_log /var/log/nginx/odoo_access.log;
    error_log /var/log/nginx/odoo_error.log;

    proxy_read_timeout 720s;
    proxy_connect_timeout 720s;
    proxy_send_timeout 720s;
    proxy_buffers 16 64k;
    proxy_buffer_size 128k;
    client_max_body_size 100m;

    gzip on;
```

```
gzip_types text/css text/plain application/json application/javascript;
```

```
# Longpolling
```

```
location /web/websocket {  
    proxy_pass http://odoo_chat;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection "Upgrade";  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

```
location /longpolling {  
    proxy_pass http://odoo_chat;  
    proxy_set_header Host $host;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

```
# Static files
```

```
location ~* /web/static/ {  
    expires 864000;  
    proxy_pass http://odoo;  
    proxy_set_header Host $host;  
}
```

```
# Main
```

```
location / {  
    proxy_pass http://odoo;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
    proxy_redirect off;  
}  
}
```

```
ln -s /etc/nginx/sites-available/odoo /etc/nginx/sites-enabled/  
nginx -t  
systemctl reload nginx
```

SSL/TLS Configuration

```
apt install -y certbot python3-certbot-nginx
```

```
certbot --nginx \
  -d erp.clientname.com \
  --non-interactive \
  --agree-tos \
  --email admin@liongate.com \
  --redirect
```

```
# Test renewal
certbot renew --dry-run
```

Domain Mapping

Same as Docker deployment - see section in `02-odoo-deployment-docker-image.md` .

Set an A record in United Domains:

- Name: erp
- Type: A
- Value: <Hetzner Floating IP>

Performance Tuning

Odoo Worker Sizing

`workers = (vCPU × 2) + 1` [theoretical max]

For CX42 (8 vCPU, 16 GB RAM):

Recommended workers: 6

max_cron_threads: 2

RAM per worker: ~400 MB

Total: 6 × 400 MB = 2.4 GB Odoo + 2 GB OS + 2 GB PostgreSQL (if local)

→ 16 GB RAM: comfortable

PgBouncer Connection Pooling (Optional)

If Odoo exhausts PostgreSQL connections, add PgBouncer:

```
apt install -y pgbouncer
```

```
# /etc/pgbouncer/pgbouncer.ini
```

```
cat > /etc/pgbouncer/pgbouncer.ini << 'EOF'
```

```
[databases]
```

```
odoo_production = host=10.0.1.20 port=5432 dbname=odoo_production
```

```
[pgbouncer]
```

```
logfile = /var/log/pgbouncer/pgbouncer.log
```

```
pidfile = /var/run/pgbouncer/pgbouncer.pid
```

```
listen_addr = 127.0.0.1
```

```
listen_port = 6432
```

```
auth_type = md5
```

```
auth_file = /etc/pgbouncer/userlist.txt
```

```
pool_mode = transaction
```

```
max_client_conn = 200
```

```
default_pool_size = 20
```

```
EOF
```

```
# Update Odoo db_host/port to use PgBouncer (port 6432)
```

Odoo Cache / Session

```
# In odoo.conf – for high-traffic, use Redis-backed sessions
# (Requires Odoo Enterprise or OCA module)
# session_db_passwd = redis://localhost:6379
```

Production Hardening

```
# 1. Restrict odoo.conf access
chown root:odoo /etc/odoo/odoo.conf
chmod 640 /etc/odoo/odoo.conf

# 2. Restrict data_dir
chown odoo:odoo /var/lib/odoo
chmod 750 /var/lib/odoo

# 3. Disable SSH root login
sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart sshd

# 4. Install security updates automatically
apt install -y unattended-upgrades
dpkg-reconfigure -plow unattended-upgrades

# 5. Fail2ban for SSH and Nginx
apt install -y fail2ban
systemctl enable fail2ban && systemctl start fail2ban

# 6. Audit open ports
ss -tlnp

# 7. Odoo: disable DB manager from internet
# Set list_db = False in odoo.conf (already set above)

# 8. Set strong master password
# admin_passwd in odoo.conf - use: openssl rand -base64 32

# 9. Nginx - restrict /web/database to internal IPs
# Add inside the server block:
#     location /web/database {
#         allow YOUR_OFFICE_IP;
#         deny all;
#         proxy_pass http://odoo;
#     }
```

Custom Addons Strategy

Directory Structure

```
/opt/odoo/custom/  
├── __init__.py          ← Not needed at root; each module is a directory  
├── custom_crm_fields/  
│   ├── __manifest__.py  
│   ├── __init__.py  
│   ├── models/  
│   ├── views/  
│   └── security/  
└── liongate_base/  
    ├── __manifest__.py  
    └── ...
```

Deploying a New Custom Module

1. Copy or clone module to /opt/odoo/custom/

```
su - odoo
```

```
git clone https://github.com/liongate/custom_crm_fields.git /opt/odoo/custom/custom_crm
```

2. Restart Odoo with module install

```
systemctl stop odoo
```

```
sudo -u odoo /opt/odoo/venv/bin/python /opt/odoo/odoo/odoo-bin \  
--config /etc/odoo/odoo.conf \  
-d odoo_production \  
-i custom_crm_fields \  
--stop-after-init
```

```
systemctl start odoo
```

3. Verify in UI:

Apps → Search for "custom_crm_fields" → Should show as installed

Updating an Existing Custom Module

```
# Pull latest code
cd /opt/odoo/custom/custom_crm_fields
git pull origin main

# Update module
systemctl stop odoo
sudo -u odoo /opt/odoo/venv/bin/python /opt/odoo/odoo/odoo-bin \
    --config /etc/odoo/odoo.conf \
    -d odoo_production \
    -u custom_crm_fields \
    --stop-after-init
systemctl start odoo
```

Backup Strategy

Backup Script

```
#!/bin/bash
# /opt/odoo/scripts/backup.sh
set -euo pipefail

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/opt/odoo/backups"
ODOO_DB="odoo_production"
DB_HOST="10.0.1.20"
DB_USER="odoo_prod"
PGPASSWORD="REPLACE"
STORAGE_BOX="uXXXXXX@uXXXXXX.your-storagebox.de"
RETENTION_DAYS=7

mkdir -p "$BACKUP_DIR"

# Dump DB
echo "[$(date)] Dumping PostgreSQL..."
PGPASSWORD="$PGPASSWORD" pg_dump \
    -h "$DB_HOST" -U "$DB_USER" -d "$ODOO_DB" \
    -Fc -f "$BACKUP_DIR/db_${TIMESTAMP}.dump"

# Backup filestore
echo "[$(date)] Backing up filestore..."
tar -czf "$BACKUP_DIR/filestore_${TIMESTAMP}.tar.gz" /var/lib/odoo/

# Backup config
cp /etc/odoo/odoo.conf "$BACKUP_DIR/odoo_conf_${TIMESTAMP}.conf"

# Sync to Storage Box
echo "[$(date)] Uploading to Storage Box..."
rsync -avz --delete \
    "$BACKUP_DIR/" \
    "${STORAGE_BOX}:/backups/odoo/" \
    -e "ssh -i /root/.ssh/storagebox_rsa -p 23"

# Cleanup old local backups
find "$BACKUP_DIR" -mtime +$RETENTION_DAYS \( -name "*.dump" -o -name "*.tar.gz" -o -na
```

```
echo "[$(date)] Backup complete: $TIMESTAMP"
```

```
chmod +x /opt/odoo/scripts/backup.sh
```

```
# Add to crontab (root)
```

```
echo "0 2 * * * /opt/odoo/scripts/backup.sh >> /var/log/odoo-backup.log 2>&1" | crontab
```

Monitoring & Logging

Log Overview

Log	Location	Command
Odoo	/var/log/odoo/odoo.log	tail -f /var/log/odoo/odoo.log
Odoo (systemd)	journald	journalctl -u odoo -f
Nginx access	/var/log/nginx/odoo_access.log	tail -f ...
Nginx error	/var/log/nginx/odoo_error.log	tail -f ...
PostgreSQL	/var/log/postgresql/	tail -f /var/log/postgresql/*.log

Basic Monitoring

```
# Install Netdata (lightweight, real-time monitoring)
```

```
curl https://get.netdata.cloud/kickstart.sh -o /tmp/netdata.sh
```

```
bash /tmp/netdata.sh --non-interactive
```

```
# Access on: http://server-ip:19999 (restrict in firewall to office IPs)
```

```
ufw allow from YOUR_OFFICE_IP to any port 19999
```

External Uptime Monitoring

- Register at uptimerobot.com (free)
- Add HTTP(S) monitor: <https://erp.clientname.com/web/health>
- Set check interval: 5 minutes

- Alert: email or Slack webhook

Upgrade Path

Patch Upgrade (same Odoo branch, e.g., 19.0.1 → 19.0.2)

```
# Pull latest 19.0 from GitHub
su - odoo
cd /opt/odoo/odoo
git pull origin 19.0

# Reinstall requirements if changed
source /opt/odoo/venv/bin/activate
pip install -r requirements.txt

# Update all modules on DB
systemctl stop odoo
sudo -u odoo /opt/odoo/venv/bin/python /opt/odoo/odoo/odoo-bin \
    --config /etc/odoo/odoo.conf \
    -d odoo_production \
    -u all \
    --stop-after-init
systemctl start odoo
```

Major Upgrade (18.0 → 19.0)

1. Create full backup (DB + filestore)
2. Clone a staging server and test the upgrade path:
 - Restore prod backup to staging
 - Set up Odoo 19.0 on staging
 - Run `odoo-upgrade` tool (requires Odoo SA subscription for Enterprise, or community migration scripts)
 - Test all modules and critical workflows
3. After staging validation, plan a maintenance window (2–6 hours)
4. Execute on production with rollback plan ready

Recovery & Restore

Full Restore Procedure

```
# Step 1: Download backup from Storage Box
rsync -avz \
    "uXXXXXX@uXXXXXX.your-storagebox.de:/backups/odoo/db_TIMESTAMP.dump" \
    /tmp/restore/
rsync -avz \
    "uXXXXXX@uXXXXXX.your-storagebox.de:/backups/odoo/filestore_TIMESTAMP.tar.gz" \
    /tmp/restore/

# Step 2: Stop Odoo
systemctl stop odoo

# Step 3: Drop and recreate DB
sudo -u postgres psql << 'EOF'
SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname='odoo_production';
DROP DATABASE IF EXISTS odoo_production;
CREATE DATABASE odoo_production OWNER odoo_prod;
EOF

# Step 4: Restore DB
PGPASSWORD="DB_PASSWORD" pg_restore \
    -h 10.0.1.20 -U odoo_prod -d odoo_production \
    /tmp/restore/db_TIMESTAMP.dump

# Step 5: Restore filestore
rm -rf /var/lib/odoo/*
tar -xzf /tmp/restore/filestore_TIMESTAMP.tar.gz -C /

# Step 6: Fix permissions
chown -R odoo:odoo /var/lib/odoo

# Step 7: Start Odoo
systemctl start odoo
systemctl status odoo

# Step 8: Verify
curl -I https://erp.clientname.com
tail -f /var/log/odoo/odoo.log
```

Validation & Testing

Startup Validation

```
# Service status
systemctl status odoo
systemctl status nginx
systemctl status postgresql # on DB server

# Odoo health endpoint
curl -s http://localhost:8069/web/health | python3 -m json.tool

# Full chain
curl -sk https://erp.clientname.com/web/health | python3 -m json.tool

# Port check
ss -tlnp | grep -E "8069|8072|443|80"

# Log check - no ERRORS should appear
tail -100 /var/log/odoo/odoo.log | grep -i error

# DB connectivity
PGPASSWORD="DB_PASSWORD" psql -h 10.0.1.20 -U odoo_prod -d odoo_production -c "\dt" | h

# Worker count
ps aux | grep odoo | grep -v grep | wc -l
```

Application Validation

1. Log in as administrator
2. Verify installed modules load without errors
3. Create a test customer record
4. Send a test email
5. Generate a PDF report (requires wkhtmltopdf)
6. Create a task in Project module
7. Check Scheduled Actions are running (Settings → Technical → Automation)

Production Deployment Checklist

Pre-Deployment

- ☐ Server provisioned (CX42 app + CX32 DB)
- ☐ Private network configured
- ☐ Firewall rules applied
- ☐ DB Volume mounted at `/mnt/pgdata`
- ☐ System dependencies installed (including wkhtmltopdf)
- ☐ Odoo system user created (`odoo`)
- ☐ Python 3.11 venv created at `/opt/odoo/venv`
- ☐ Odoo source cloned at `/opt/odoo/odoo`
- ☐ Python requirements installed
- ☐ PostgreSQL installed and tuned on DB server
- ☐ `odoo_prod` DB user and `odoo_production` database created
- ☐ `pg_hba.conf` allows app server IP
- ☐ `/etc/odoo/odoo.conf` created with production values
- ☐ `list_db = False` confirmed
- ☐ Strong `admin_passwd` set
- ☐ SMTP configured

Deployment

- ☐ `systemd` service file created
- ☐ `systemctl enable odoo` done
- ☐ `systemctl start odoo` - service starts cleanly
- ☐ `journalctl -u odoo` shows no errors
- ☐ Nginx installed and configured
- ☐ `nginx -t` passes
- ☐ SSL certificate obtained via Certbot
- ☐ HTTPS access working
- ☐ HTTP → HTTPS redirect working
- ☐ DNS A record pointing to Floating IP

Post-Deployment

- ☐ Admin password changed
- ☐ Developer mode disabled

- ☐ DB manager restricted (`list_db = False`)
- ☐ Backup script deployed and tested
- ☐ Cron job scheduled for daily backup
- ☐ First backup verified on Storage Box
- ☐ Restore tested on separate test environment
- ☐ Uptime monitoring configured
- ☐ Log rotation configured
- ☐ SSL auto-renewal verified
- ☐ Fail2ban running
- ☐ SSH root login disabled
- ☐ Odoo scheduled actions reviewed

Document owner: LIONGATE SARL DevOps Team

Last updated: See Git history

Next review: After each Odoo source upgrade